Informe del Laboratorio 01

Primeros pasos con Python - Cotizador de compra

Aprendiz	Anthony Córdova Calderón
Ficha	3388087
Programa	ADSO
Laboratorio	Laboratorio 01 de Python: variables, tipos de datos, operadores, entrada con input() y actividad integradora (Cotizador de compra)
Fecha del informe	21 de octubre de 2026
Repositorio de GitHub	https://github.com/skyyleerr/python-laboratorios
Carpeta del laboratorio	laboratorio-01/

1. Descripción de las actividades y funcionalidades desarrolladas

Todos los programas están en la carpeta **laboratorio-01** del repositorio y se ejecutan con `python archivo.py`.

Práctica	Archivos	Qué hace / qué se practicó
1.1 Entorno	hola_python.py, presentacion.py, laboratorio_de_errores.py	Verificar Python, ejecutar un .py desde VS Code y terminal, presentación con print() y respuestas de comprobación. Se provocó un ValueError para leer el mensaje de error.
1.2 Variables y tipos	datos_basicos.py, perfil_aprendiz.py, inventario.py, error_variables.py	Variables str/int/float/bool, type(), f-strings, reasignación (Monitor: 8 a 5 unidades, $720000 a $699000), constantes (IMPUESTO) y depuración de un programa con errores.
1.3 Operadores	operadores.py, conversion_tiempo.py, calculo_promedio.py, calculo_compra.py, calculo_salario.py, factura_simple.py	Operadores + - * / // % **, orden de operaciones, horas/minutos con // y %, promedio de notas, compra con descuento, salario con aporte del 4 %, factura con descuento e impuesto.
1.4 input()	saludo.py, tipo_entrada.py, prueba_entrada.py, datos_usuario.py, suma_interactiva.py, conversor_temperatura.py, salario_interactivo.py, programa_incompleto.py, conversion_unidades.py	Lectura de texto, conversión con int()/float(), ficha de usuario, calculadora interactiva, Celsius a Fahrenheit, resumen de pago, subtotal interactivo y km a metros/cm/millas.
1.5 Integrador	cotizador_compra.py, pruebas_cotizador.py, cotizador_ampliado.py, cotizador_depuracion.py	Cotizador de consola (cliente, producto, precio, cantidad, % descuento, % impuesto) con subtotal, descuento, base, impuesto y total; pruebas automáticas; ampliación con envio, vendedor y código; reto de depuración.

Funcionamiento del cotizador

- **subtotal** = precio_unitario * cantidad
- **valor_descuento** = subtotal * (descuento / 100)
- **base** = subtotal - valor_descuento
- **valor_impuesto** = base * (impuesto / 100): el impuesto se aplica sobre la base, como pide el enunciado
- **total** = base + valor_impuesto. Cada resultado se guarda primero en una variable y después se imprime.

2. Estado del trabajo

Actividades completadas

- Prácticas 1.1, 1.2, 1.3 y 1.4 con todos sus ejercicios y retos.
- Cotizador de compra completo y probado (5 de 5 casos correctos).
- Retos de ampliación (envio, vendedor, código) y de depuración (tres errores provocados y corregidos).
- README del laboratorio, repositorio con commits progresivos y este informe.

Actividades pendientes

- Ninguna actividad del laboratorio queda pendiente. Los laboratorios 02, 03 y 04 usaran este mismo repositorio.
- El manejo de excepciones (try/except) para entradas inválidas se trabaja más adelante en el curso; en este laboratorio se analiza como respuesta a la pregunta 2.

Dificultades y soluciones aplicadas

Dificultad	Solución aplicada
El cotizador mostraba `Subtotal: $ 360000.0` (con espacio) y el enunciado pide `$360000.0`.	Las pruebas automáticas detectaron la diferencia (0 de 5). Se reemplazó print("...: $", x) por f-strings f"...: ${x}" (commit 04612ee).
inventario.py no imprimía el estado inicial ni el actualizado.	Se agregaron los print() de ambos momentos con un título para distinguirlos (commit 65b0ede).
Error de ortografía en presentacion.py ("estudianto").	Se corrigió a "estudiando" (commit 8388ffb).
Confusión entre la entrada como texto y como número.	Se convierte con int()/float() inmediatamente después de input() y se comprueba con type().
Porcentajes enteros (10 = 10 %).	Se dividen entre 100 antes de multiplicar.

3. Seguimiento de commits relevantes

Historial verificable con `git log --oneline` en el repositorio. Cada fila explica qué se modificó, por qué y cuál fue el resultado.

Commit	Mensaje	Qué se modificó	Por qué	Resultado
fc3e4a9	chore: crear repositorio...	Se creó el repositorio con README general y .gitignore.	Base única para los cuatro laboratorios.	Repositorio listo.
fdfc96c	feat(1.1): primer programa, presentación y laboratorio de errores	Se agregaron hola_python.py, presentacion.py y laboratorio_de_errores.py.	Verificar el entorno y practicar la lectura de errores.	Programas ejecutables en VS Code y terminal.
7d87655	feat(1.2): variables, tipos de datos...	Se agregaron datos_basicos.py, perfil_aprendiz.py y error_variables.py.	Practicar variables, type(), f-strings y depuración.	Salidas con el formato pedido.
c24358e	feat(1.2): primera versión de inventario.py	Se agregó inventario.py con la reasignación de valores.	Cubrir el ejercicio de estado del inventario.	Versión inicial; sin salida por pantalla (se corrige después).
79f322e	feat(1.3): operadores, conversión de tiempo...	Se agregaron 6 archivos de operadores y cálculos.	Practicar + - * / // % **, orden de operaciones y cálculos por etapas.	Resultados verificados a mano (p. ej. 367 min = 6 h 7 min).
8f89616	feat(1.4): entrada de datos con input()...	Se agregaron 9 programas interactivos.	Pasar de valores fijos a datos del usuario con int()/float().	Programas interactivos funcionando.
22a913c	feat(1.5): primera versión del cotizador	Se agregó cotizador_compra.py con los cinco cálculos.	Implementar el reto integrador por etapas.	Totales correctos, formato de salida con diferencias.
4f97bc0	test(1.5): pruebas automáticas	Se agregó pruebas_cotizador.py (ejemplo + casos 1-4).	Comprobar el cotizador con los casos del laboratorio sin teclear datos cada vez.	Resultado inicial: 0 de 5, por el formato "$ ".
04612ee	fix(1.5): formato del enunciado	Se cambiaron los print() por f-strings y se documentó el orden de cálculo en comentarios.	La salida debía ser $180000.0 y no $ 180000.0.	5 de 5 casos correctos.
65b0ede	fix(1.2): inventario.py	Se agregaron los print() del estado inicial y actualizado.	El ejercicio pide distinguir ambos momentos.	Salida con "ESTADO INICIAL" y "ESTADO ACTUALIZADO".
8388ffb	fix(1.1): ortografía en presentacion.py	estudianto a estudiando.	Presentación sin errores.	Texto corregido.
7ef187f	feat(1.5): reto de ampliación	Se agregó cotizador_ampliado.py (código, vendedor, envio).	El envio se suma después del impuesto.	Total de ejemplo: $400560.0.
280b8b3	feat(1.5): reto de depuración	Se agregó cotizador_depuracion.py con tres errores comentados.	Documentar error de sintaxis, de nombre y lógico.	Errores explicados y corregidos.
262b444	docs: README del laboratorio	Se agregó laboratorio-01/README.md.	Describir el proyecto, ejecución y pruebas.	Instrucciones completas.
(último)	docs: agregar informe PDF	Se agregó informe/Informe_Laboratorio_01.pdf.	Entregar el informe junto al código.	Repositorio completo.

4. Evidencias de las pruebas realizadas

Se ejecutó `python pruebas_cotizador.py`, que corre el cotizador con cada caso y compara con lo esperado.

Caso	Entrada (precio / cant. / desc. % / imp. %)	Esperado	Obtenido	Estado
Ejemplo del enunciado	180000 / 2 / 10 / 19	TOTAL $385560.0	TOTAL $385560.0	Correcto
Caso 1	100000 / 2 / 10 / 0	180000.0	180000.0	Correcto
Caso 2	50000 / 3 / 0 / 0	150000.0	150000.0	Correcto
Caso 3	80000 / 5 / 25 / 0	300000.0	300000.0	Correcto
Caso 4 (cálculo manual: 100000 + 19 % = 119000)	100000 / 1 / 0 / 19	119000.0	119000.0	Correcto

Antes de la corrección (commit 4f97bc0):

```
[FALLA] Ejemplo del enunciado
        esperado: 'Precio unitario: $180000.0'
        esperado: 'Subtotal: $360000.0'
        esperado: 'Descuento: $36000.0'
        esperado: 'Base: $324000.0'
        esperado: 'Impuesto: $61560.0'
        esperado: 'TOTAL: $385560.0'
[FALLA] Caso 1
        esperado: 'TOTAL: $180000.0'
[FALLA] Caso 2
        esperado: 'TOTAL: $150000.0'
[FALLA] Caso 3
        esperado: 'TOTAL: $300000.0'
[FALLA] Caso 4
        esperado: 'TOTAL: $119000.0'

0/5 casos correctos
```

Después de la corrección (commit 04612ee):

```
[OK ] Ejemplo del enunciado
[OK ] Caso 1
[OK ] Caso 2
[OK ] Caso 3
[OK ] Caso 4

5/5 casos correctos
```

Ejecución del cotizador con el ejemplo del enunciado:

```
============================
      COTIZADOR PYTHON
============================

----------------------------
      RESUMEN DE COMPRA
----------------------------
Cliente: Laura
Producto: Teclado mecánico
Precio unitario: $180000.0
Cantidad: 2
Subtotal: $360000.0
Descuento: $36000.0
Base: $324000.0
Impuesto: $61560.0
TOTAL: $385560.0
```

Otras comprobaciones:

- **Inventario** (inventario.py): Unidades 8 a 5, Precio $720000.0 a $699000.0.
- **Ampliación** (cotizador_ampliado.py): con envio $15000 el total es $400560.0 (385560 + 15000).
- **Depuración**: error de sintaxis (`SyntaxError: '(' was never closed`), error de nombre (`NameError: name 'totall' is not defined`) y error lógico (200000 - 200000*10 =

-1800000.0 en lugar de 180000.0), corregidos en cotizador_depuracion.py.

5. Preguntas de análisis situacional

Pregunta 1

¿Qué harías si el programa muestra un error porque intentas utilizar una variable que no ha sido declarada o cuyo nombre está escrito incorrectamente?

Qué haría. Leería el mensaje de error completo, empezando por la última línea, que indica el tipo (NameError) y el nombre que Python no reconoce, y la línea del traceback que señala dónde ocurre. Después buscaría dónde se creó esa variable.

Por qué. Python distingue mayúsculas de minúsculas y solo conoce los nombres que ya fueron asignados antes de usarse. Un NameError casi siempre es un error de escritura (mensage en vez de mensaje, Total en vez de total) o una variable usada antes de asignarla. El mensaje suele sugerir el nombre parecido ("Did you mean: 'total'?").

Cómo lo implementaría. Revisaría: (1) que el nombre del print o de la operación sea idéntico al de la asignación, letra por letra y con las mismas mayúsculas; (2) que la asignación este antes del uso y en el mismo archivo; (3) que no se haya borrado o renombrado la línea que crea la variable. Corregiría solo ese nombre, sin cambiar otras líneas a la vez.

```
mensaje = "Hola"
print(mensage)   # NameError: name 'mensage' is not defined
print(mensaje)   # corregido
```

Cómo comprobaría el resultado. Volvería a ejecutar el archivo: ya no debe aparecer el NameError y la salida debe coincidir con la esperada. Lo hice en el laboratorio con mensage/mensaje y con totall/total (cotizador_depuracion.py), y con python pruebas_cotizador.py se comprueba que el cotizador completo sigue en 5 de 5.

Pregunta 2

¿Qué harías si un usuario introduce letras cuando el cotizador solicita un precio mediante input()?

Qué haría. Identificaría que el programa termina con ValueError y evitaría que se cierre de forma inesperada validando la entrada antes de usarla.

Por qué. input() siempre devuelve texto y float("abc") no puede convertirlo a número, por eso lanza ValueError: could not convert string to float. En el laboratorio se vio igual con int("veinte") en laboratorio_de_errores.py. Como el programa no controla el error, Python lo detiene y el usuario pierde sus datos.

Cómo lo implementaría. Alternativas: (a) try/except ValueError dentro de un bucle while que vuelve a pedir el dato hasta que sea válido; (b) validar antes con texto.replace('.', '', 1).isdigit(); (c) rechazar valores negativos o cero con una condición. La opción recomendada es try/except en una función reutilizable, que se vera formalmente más adelante en el curso.

```
def pedir_precio():
    while True:
        texto = input("Precio unitario: ")
        try:
            valor = float(texto)
            if valor > 0:
                return valor
            print("El precio debe ser mayor que cero.")
        except ValueError:
            print("Escribe solo números, por ejemplo 85000.")

precio_unitario = pedir_precio()
```

Cómo comprobaría el resultado. Probaría tres entradas: un número válido (100000), letras (abc) y un valor vacío. Con el código original abc detiene el programa; con la validación el programa muestra un aviso y vuelve a preguntar. Además ejecutaría otra vez pruebas_cotizador.py para confirmar que los casos válidos no cambiaron.

Pregunta 3

¿Qué harías si el cotizador calcula correctamente el subtotal, pero el descuento aplicado no corresponde al porcentaje indicado por el usuario?

Qué haría. Comprobaría el descuento con un caso de cálculo manual sencillo y revisaría la línea del descuento, el orden de los cálculos y las variables que usa.

Por qué. Si el subtotal está bien, el fallo está en la línea del descuento o después. Las causas más probables son olvidar dividir el porcentaje entre 100 (10 en vez de 0.10), usar la variable equivocada (por ejemplo precio_unitario en lugar de subtotal) o calcular el descuento después del impuesto.

Cómo lo implementaría. Imprimiría temporalmente los valores intermedios (subtotal, descuento_decimal, valor_descuento) y comprobaría el orden correcto: subtotal, valor_descuento = subtotal * (descuento / 100), base = subtotal - valor_descuento, y solo entonces el impuesto sobre la base. Compararía cada valor con la calculadora y quitaría los print de depuración al terminar.

```
descuento = 10
valor_descuento = subtotal * descuento          # incorrecto: 2000000.0
valor_descuento = subtotal * (descuento / 100)  # correcto: 20000.0
```

Cómo comprobaría el resultado. Con precio 100000, cantidad 2 y descuento 10 el descuento debe ser 20000.0 y el total 180000.0. Con el error de no dividir entre 100 el resultado era -1800000.0, evidenciando el fallo. Después de corregir ejecutó los casos 1 a 4 de pruebas_cotizador.py y deben dar 5 de 5.

Pregunta 4

¿Qué harías si necesitas calcular el promedio de varios valores y el resultado obtenido no coincide con el cálculo manual?

Qué haría. Haría el cálculo manual paso a paso, lo compararía con cada paso del programa y revisaría los operadores, los tipos y el orden de las operaciones.

Por qué. La causa típica es la precedencia: la división se ejecuta antes que la suma, por lo que nota_1 + nota_2 + nota_3 / 3 solo divide la última nota. También pueden fallar los tipos (notas leídas con input() sin convertir, que no se pueden sumar como números) o dividir entre una cantidad equivocada.

Cómo lo implementaría. Pondría paréntesis alrededor de la suma, guardaría la suma en una variable intermedia (como en calculo_promedio.py: resultado = nota_1 + nota_2 + nota_3 y luego promedio = resultado / 3), verificaría con type() que las notas sean float y usaría / (no //) para conservar decimales.

```
nota_1, nota_2, nota_3 = 4.2, 3.8, 4.5
print(nota_1 + nota_2 + nota_3 / 3)    # 9.5  (incorrecto)
print((nota_1 + nota_2 + nota_3) / 3)  # 4.1666666666666667 (correcto)
```

Cómo comprobaría el resultado. Con 4.2, 3.8 y 4.5 la suma es 12.5 y el promedio es 4.1666..., y sin paréntesis el programa da 9.5. Comprobaría que el resultado coincida con la calculadora y probaría con otros valores, incluidos iguales (3, 3, 3 debe dar 3.0).

Pregunta 5

¿Qué harías si después de modificar el cotizador deja de funcionar una operación que anteriormente producia el resultado correcto?

Qué haría. Usaría el historial de commits para encontrar el cambio que introdujo el fallo, corregirlo y dejar la solución documentada en un commit.

Por qué. Como cada cambio queda en un commit pequeño con mensaje descriptivo, se puede comparar la versión que funcionaba con la que falla y ubicar exactamente qué línea cambió, en lugar de buscar a ciegas.

Cómo lo implementaría. (1) git log --oneline para ver los commits y elegir el último que funcionaba; (2) git diff COMMIT_BUENO COMMIT_MALO -- laboratorio-01/cotizador_compra.py para ver las líneas modificadas; (3) si hay muchos commits, git bisect para hallar el responsable; (4) corregir el código (o git revert COMMIT si todo el cambio fue un error); (5) commit con mensaje fix: que diga qué fallaba, por qué y cómo se corrigió; (6) git push, de modo que GitHub conserve el historial. En GitHub se puede hacer lo mismo en la pestaña Commits.

```
git log --oneline
git diff 22a913c 04612ee -- laboratorio-01/cotizador_compra.py
git commit -m "fix: corregir descuento que no dividía entre 100"
```

Cómo comprobaría el resultado. Ejecutaría python pruebas_cotizador.py: deben pasar los 5 casos. Este flujo se aplico en este laboratorio: el commit 4f97bc0 agregó las pruebas y estas fallaron (0 de 5) por el formato; el commit 04612ee lo corrigió y las pruebas pasaron (5 de 5).

6. Conclusion

El repositorio contiene todos los ejercicios y el cotizador funcionando. Las pruebas del laboratorio (casos 1 a 4 y el ejemplo del enunciado) pasan 5 de 5, y el historial de commits permite verificar cada cambio descrito en este informe.